

NSW Task Flow: Data-Driven Orchestration Architecture

This document outlines our orchestration architecture. To maximize reusability and avoid over-engineering, the system relies on **Data-Driven Workflows**.

Instead of writing custom Go code for every new permit or process, we author JSON configurations that compose generic execution blocks.

1. Architectural Overview

The architecture is divided into three conceptual layers, bound together by a **Task Template Registry**.

Both Layer 1 and Layer 2 use the exact same WorkflowDefinition graph structures (Nodes, Edges, Gateways), executed by the GraphInterpreterWorkflow.

- **Layer 1 (The Macro Journey):** A JSON definition of the high-level business milestones.
- **The Task Registry:** A configuration map linking Layer 1 tasks to Layer 2 workflows and UI rendering data.
- **Layer 2 (The Micro Flow):** A JSON definition of a specific task loop, composing generic actions.
- **Layer 3 (The Capabilities Library):** The actual Go code (Temporal Activities) that execute specific types of work via TaskManager Plugins.

2. The Task Template Registry (The Bridge)

When Layer 1 defines a node, it specifies a task_template_id. The **TaskManager (TM)** looks this ID up in a registered manifest.

This manifest defines exactly how the TM should initialize the task, what sub-workflow to run, and what UI schemas are required.

Example Registry Entry: phyto_application_task

```
{
  "template_id": "phyto_application_task",
  "task_type": "APPLICATION",
  "workflow_id": "Phyto_Application_Flow_v1",
  "user_jsonforms_id": "phyto_user_form_v1",
  "reviewer_jsonforms_id": "phyto_reviewer_form_v1"
}
```

The TaskManager Plugin Architecture

The TaskManager uses the `task_type` (e.g., APPLICATION) to route execution to registered plugins.

When an APPLICATION task starts, the TM generates a unique **TaskID**. It creates a single Database entry keyed by this TaskID, storing the JSONForms IDs, the initialized data, and the execution coordinates. It then starts the Layer 2 GraphInterpreterWorkflow.

3. Layer 3: The Capabilities Library (The Code)

Layer 3 Temporal Activities are executed by the generic GraphInterpreterWorkflow. They interact directly with the single DB entry for the task.

- `generic_user_input`: Updates the DB entry to indicate the task is waiting on the user. Records the current NodeID as the active Temporal coordinate. Suspends execution.
- `generic_external_review`: Updates the DB entry to indicate the task is queued externally. Records the current NodeID. Suspends execution.

Crucially, **the data passed through the workflow exactly matches the namespaced jsonforms schema**. This eliminates the need for complex data mapping; the workflow state natively represents the UI state.

4. Layer 2: The Micro Flow (The JSON Template)

Layer 2 defines a reusable sub-workflow using purely JSON. It orchestrates the Layer 3 building blocks.

Notice how `input_mapping` and `output_mapping` pass the entire namespaced userform and reviewerform objects. This keeps the global state impeccably organized. The Gateway evaluates conditions directly against the reviewerform namespace.

```
{
  "id": "Phyto_Application_Flow_v1",
  "name": "Phyto Sanitary Application Sub-Process",
  "version": 1,
  "nodes": [
    { "id": "start", "type": "START" },
    {
      "id": "user_submission",
      "type": "TASK",
      "task_template_id": "generic_user_input",
      "input_mapping": {
        "userform": "userform"
      },
    },
  ],
}
```

```

    "output_mapping": {
      "userform": "userform"
    }
  },
  {
    "id": "external_review",
    "type": "TASK",
    "task_template_id": "generic_external_review",
    "input_mapping": {
      "userform": "userform",
      "reviewerform": "reviewerform"
    },
    "output_mapping": {
      "reviewerform": "reviewerform"
    }
  },
  {
    "id": "review_gateway",
    "type": "GATEWAY",
    "gateway_type": "EXCLUSIVE_SPLIT"
  },
  { "id": "end", "type": "END" }
],
"edges": [
  { "id": "e1", "source_id": "start", "target_id": "user_submission" },
  { "id": "e2", "source_id": "user_submission", "target_id": "external_review" },
  { "id": "e3", "source_id": "external_review", "target_id": "review_gateway" },
  {
    "id": "e_approve",
    "source_id": "review_gateway",
    "target_id": "end",
    "condition": "reviewerform.review_outcome == 'approve'"
  },
  {
    "id": "e_needs_info",
    "source_id": "review_gateway",
    "target_id": "user_submission",
    "condition": "reviewerform.review_outcome == 'needs_more_info'"
  }
]
}

```

5. Layer 1: The Macro Journey (The JSON Process)

Layer 1 strings together tasks to define an end-to-end business process. It treats tasks as black boxes.

Here we map the initial `applicant_name` (provided when the macro journey started) directly into the `userform.applicant_name` namespace for the `phase1_app` task.

```
{
  "id": "Get_Phyto_Permit",
  "name": "End-to-End Phyto Permit Journey",
  "version": 1,
  "nodes": [
    { "id": "start", "type": "START" },
    {
      "id": "phase1_app",
      "type": "TASK",
      "task_template_id": "phyto_application_task",
      "input_mapping": {
        "applicant_name": "userform.applicant_name"
      },
      "output_mapping": {
        "reviewerform.review_outcome": "phase1_outcome"
      }
    },
    { "id": "end", "type": "END" }
  ],
  "edges": [
    { "id": "e1", "source_id": "start", "target_id": "phase1_app" },
    { "id": "e2", "source_id": "phase1_app", "target_id": "end" }
  ]
}
```

6. End-to-End State Execution Flow (Single DB Truth & Unified Endpoint)

The core principle is that **Layer 2 Temporal Global State** exactly mirrors the namespaced **JSONForms structure**, while a **single DB entry keyed by a unique TaskID** acts as a materialized view for the UI and the routing map for callbacks.

1. **Layer 1 Triggers TaskManager:** TM looks up `phyto_application_task` in the registry. It maps `applicant_name` into the `userform` object namespace.

2. **TaskID Generation & DB Setup:** TM generates a TaskID. It creates a *single* DB record containing the JSONForms IDs, the Layer 1 identifiers, and the initialized userform data. It starts the Layer 2 Workflow.
3. **Generic User Input Node:** The node updates the DB entry, saving its NodeID as the ActiveActivityID. It maps the userform object into the node inputs. It goes to sleep.
4. **UI Render & Unified Submit:** The Portal reads the DB via `/api/task/{id}`, renders the form (pre-filling `userform.applicant_name`), and the user fills the rest of the fields before submitting to `/api/task/{id}`.
5. **API Validation & Wakeup:** The TM's APPLICATION handler validates the submission, updates the DB entry, and uses the stored ActiveActivityID to wake up the exact Temporal node.
6. **Global State Merge:** The workflow's `output_mapping` merges the submitted userform object directly into Layer 2's global state.
7. **Generic External Review Node:** The next node takes the userform and reviewerform objects, triggers the external API, updates the DB to "Queued", sets the new ActiveActivityID, and sleeps, waiting for the external system to call `/api/task/{id}`.

7. Implementation: Pseudo-Code & Execution Trace

To ensure Layer 1 and Layer 2 coordinates do not overwrite each other, the single DB entry stores them in distinct fields.

The Database Schema Definition

```
type TaskRecord struct {
    TaskID      string
    TaskType    string
    UserFormID   string
    ReviewerFormID string
    Status      string // Used for UI Rendering ("PENDING_USER", "QUEUED_EXTERNALLY") and
    API validation

    // Parent Coordinates (Used to wake up Layer 1 when Layer 2 finishes)
    Layer1WorkflowID string
    Layer1RunID      string
    Layer1NodeID     string

    // Active Sub-Process Coordinates (Used to wake up Layer 2 during user/API interaction)
    Layer2WorkflowID string
    Layer2RunID      string
    ActiveActivityID  string

    Data      map[string]any // Mirrors JSONForms strictly
}
```

The Temporal Activity Adapter

Inside the generic `GraphInterpreterWorkflow`, when a TASK node is reached, it executes a Temporal Activity. This activity bridges the gap between Temporal's internal context and our external `TaskManager`.

```
// This is the generic Activity registered with the Temporal Worker
func ExecuteTaskActivity(ctx context.Context, taskTemplateID string, inputs map[string]any)
(map[string]any, error) {
    info := activity.GetInfo(ctx)

    // Bundle Temporal's internal context into the custom TaskPayload
    payload := TaskPayload{
        WorkflowID: info.WorkflowExecution.ID,
        RunID:      info.WorkflowExecution.RunID,
        NodeID:     info.ActivityID, // Mapped to the graph's Node ID
        TaskTemplateID: taskTemplateID,
        Inputs:     inputs,
    }

    // Call the TaskManager!
    // This will return activity.ErrResultPending to pause the workflow asynchronously.
    return GlobalTaskManager.HandleTask(payload)
}
```

The TaskManager Handlers & DB Setup

When Layer 1 starts the task, the TM sets up the central routing record, defining the Layer 1 parent coordinates.

```
// Bound to Layer 1 Worker's TaskActivationHandler
func (tm *TaskManager) StartTask(payload TaskPayload) error {
    registryData := tm.Registry.Get(payload.TaskTemplateID)
    taskID := generateTaskID()

    // 1. Create the Single DB Record, setting the Parent Coordinates
    tm.DB.CreateTask(TaskRecord{
        TaskID:      taskID,
        TaskType:    registryData.TaskType,
        UserFormID:  registryData.UserJsonFormsID,
        ReviewerFormID: registryData.ReviewerJsonFormsID,
    })
}
```

```

        Status:      "STARTING",
        // Parent Coordinates
        Layer1WorkflowID: payload.WorkflowID,
        Layer1RunID:    payload.RunID,
        Layer1NodeID:   payload.NodeID,
    })

    // 2. Trigger Layer 2, injecting the mapped Layer 1 inputs
    initialState := payload.Inputs
    initialState["_task_id"] = taskID
    tm.Layer2Manager.StartWorkflow(context.Background(), registryData.WorkflowID,
    initialState)

    return activity.ErrResultPending
}

```

Inside Layer 2, generic capabilities update the Active Sub-Process coordinates, leaving the parent coordinates intact.

// Inside GlobalTaskManager.HandleTask(payload) triggered by Layer 3 Activity Adapter

```

func (tm *TaskManager) HandleTask(payload TaskPayload) (map[string]any, error) {

```

```

    taskID := payload.Inputs["_task_id"].(string)

```

```

    switch payload.TaskTemplateID {
    case "generic_user_input":
        tm.DB.UpdateTask(taskID, TaskUpdate{
            Status:      "PENDING_USER", // UI knows to render editable form
            // Active Sub-Process Coordinates
            Layer2WorkflowID: payload.WorkflowID,
            Layer2RunID:    payload.RunID,
            ActiveActivityID: payload.NodeID,
            Data:           payload.Inputs, // Sync UI data
        })
        return nil, activity.ErrResultPending // Suspend

```

```

    case "generic_external_review":
        tm.DB.UpdateTask(taskID, TaskUpdate{
            Status:      "QUEUED_EXTERNALLY", // UI knows to render read-only
            // Active Sub-Process Coordinates
            Layer2WorkflowID: payload.WorkflowID,
            Layer2RunID:    payload.RunID,
            ActiveActivityID: payload.NodeID,

```

```

        Data:      payload.Inputs,
    })
    // Dispatch to external system...
    return nil, activity.ErrResultPending // Suspend
}
}

```

Unpausing the Workflow (The Unified Completion API)

The API wakes up Layer 2 using the active coordinates.

```

// Endpoint: POST /api/task/{taskID}
func (api *Server) handleTaskInteraction(taskID string, payload map[string]any) {
    // 1. Fetch the single DB entry mapping
    taskRecord := api.DB.GetTask(taskID)
    if taskRecord == nil {
        return NotFound()
    }

    // 2. Route to Task-Type Specific Handler for Validation
    // The handler enforces valid Status transitions (e.g., verifying it's "PENDING_USER" or
    "QUEUED_EXTERNALLY")
    // and validates the payload against the jsonforms schema.
    handler := api.TaskManager.GetPluginHandler(taskRecord.TaskType)
    if err := handler.ValidateAndProcess(taskRecord, payload); err != nil {
        return BadRequest(err)
    }

    // 3. Update the DB with the new validated data
    api.DB.UpdateTaskData(taskID, payload)

    // 4. WAKE UP TEMPORAL WORKFLOW
    // The payload exactly matches the namespaced jsonforms schema.
    api.TemporalClient.CompleteActivityByID(
        context.Background(),
        "default",
        taskRecord.Layer2WorkflowID,
        taskRecord.Layer2RunID,
        taskRecord.ActiveActivityID, // E.g., 'user_submission' or 'external_review'
        payload,
        nil,
    )
}

```



```
}
```

Completing the Process (Waking up Layer 1)

When Layer 2 hits the END node, the WorkflowCompletedActivity handler fires. This uses the parent coordinates stored in the DB to wake Layer 1 back up.

```
// WorkflowCompletionHandler for Layer 2
func (tm *TaskManager) OnLayer2Complete(layer2WorkflowID string, finalGlobalState
map[string]any) error {
    taskRecord := tm.DB.GetTaskByLayer2WorkflowID(layer2WorkflowID)

    tm.DB.UpdateStatus(taskRecord.TaskID, "COMPLETED") // UI puts task in History

    // Map outputs from Layer 2 global state for Layer 1
    registryData := tm.Registry.Get(taskRecord.TaskTemplateID)
    mappedOutput := mapOutputs(finalGlobalState, registryData.WorkflowOutputs)

    // WAKE UP LAYER 1 TEMPORAL WORKFLOW using Parent Coordinates
    return tm.Layer1Manager.TaskDone(
        context.Background(),
        taskRecord.Layer1WorkflowID,
        taskRecord.Layer1RunID,
        taskRecord.Layer1NodeID,
        mappedOutput,
    )
}
```

8. UI Templates & Namespaced State Mirroring (JSONForms)

Because our workflow state natively mirrors the JSONForms structure, we use JSON Schema's nested objects (properties.userform and properties.reviewerform) to enforce strict namespacing.

The Namespaced Data Flow Example

1. **Layer 1** starts the task, mapping applicant_name into the userform object.
2. **Layer 2's** global state looks like: {"_task_id": "123", "userform": {"applicant_name": "Acme Farms Ltd."}}.
3. **User Form Render:** JSONForms automatically pre-fills the

#/properties/userform/properties/applicant_name field.

4. **User Submit:** The user submits. The payload is naturally namespaced: {"userform": {"applicant_name": "Acme Farms Ltd.", "crop_type": "Apples"}}. The workflow captures this entire object.
5. **Reviewer Render:** The reviewer form schema includes both userform (read-only) and a new reviewerform object. The reviewer submits {"reviewerform": {"review_outcome": "approve"}}.

8.1 User Form (phyto_user_form_v1)

This is the form presented to the applicant via the User Portal. We wrap all fields inside a userform object in the JSON Schema.

```
{
  "schema": {
    "type": "object",
    "properties": {
      "userform": {
        "type": "object",
        "properties": {
          "applicant_name": { "type": "string", "title": "Registered Applicant Name" },
          "farm_location": { "type": "string", "title": "Farm Address / Location" },
          "crop_type": { "type": "string", "title": "Commodity / Crop Type" },
          "export_destination": { "type": "string", "title": "Destination Country" }
        },
        "required": ["farm_location", "crop_type", "export_destination"]
      }
    }
  },
  "uischema": {
    "type": "VerticalLayout",
    "elements": [
      {
        "type": "Group",
        "label": "Applicant Details",
        "elements": [
          {
            "type": "Control",
            "scope": "#/properties/userform/properties/applicant_name",
            "options": {
              "readonly": true,
              "detail": "This is automatically populated from your profile."
            }
          }
        ]
      }
    ]
  }
}
```

```

    ]
  },
  {
    "type": "Group",
    "label": "Consignment Details",
    "elements": [
      {
        "type": "Control",
        "scope": "#/properties/userform/properties/farm_location"
      },
      {
        "type": "HorizontalLayout",
        "elements": [
          {
            "type": "Control",
            "scope": "#/properties/userform/properties/crop_type"
          },
          {
            "type": "Control",
            "scope": "#/properties/userform/properties/export_destination"
          }
        ]
      }
    ]
  }
]
}
]
}
}

```

8.2 Reviewer Form (phyto_reviewer_form_v1)

The Reviewer form defines both the userform (read-only context) and the reviewerform (actionable outcome). The string values for review_outcome (approve, reject, needs_more_info) match the Condition expressions in the Layer 2 Gateway node!

```

{
  "schema": {
    "type": "object",
    "properties": {
      "userform": {
        "type": "object",
        "properties": {

```

```

    "applicant_name": { "type": "string", "title": "Applicant Name" },
    "farm_location": { "type": "string", "title": "Farm Address" },
    "crop_type": { "type": "string", "title": "Commodity" },
    "export_destination": { "type": "string", "title": "Destination" }
  }
},
"reviewerform": {
  "type": "object",
  "properties": {
    "review_outcome": {
      "type": "string",
      "title": "Review Decision",
      "enum": ["approve", "reject", "needs_more_info"]
    },
    "reviewer_comments": {
      "type": "string",
      "title": "Comments / Justification"
    }
  },
  "required": ["review_outcome"]
}
},
"uischema": {
  "type": "VerticalLayout",
  "elements": [
    {
      "type": "Group",
      "label": "Submitted Application Data (Read Only)",
      "elements": [
        {
          "type": "Control",
          "scope": "#/properties/userform/properties/applicant_name",
          "options": { "readonly": true }
        },
        {
          "type": "Control",
          "scope": "#/properties/userform/properties/farm_location",
          "options": { "readonly": true }
        },
        {
          "type": "HorizontalLayout",
          "elements": [

```

```

    {
      "type": "Control",
      "scope": "#/properties/userform/properties/crop_type",
      "options": { "readonly": true }
    },
    {
      "type": "Control",
      "scope": "#/properties/userform/properties/export_destination",
      "options": { "readonly": true }
    }
  ]
}
]
},
{
  "type": "Group",
  "label": "Official Review",
  "elements": [
    {
      "type": "Control",
      "scope": "#/properties/reviewerform/properties/review_outcome",
      "options": {
        "format": "radio"
      }
    },
    {
      "type": "Control",
      "scope": "#/properties/reviewerform/properties/reviewer_comments",
      "options": {
        "multi": true,
        "placeholder": "Provide feedback if 'Needs More Info' or 'Reject' is selected..."
      }
    }
  ]
}
]
}
]
}
}

```